EMG GENIECIVIL INDUSTRIEL ELECTRIQUE INFORMATIQUE
المدرسة المغربية للهندسة
ECOLE MAROCAINE D'INGENIERIE

Automatisation CI/CD avec Docker et GitHub Actions

Rapport de Mini-Projet DevOps : L'approche Everything as Code

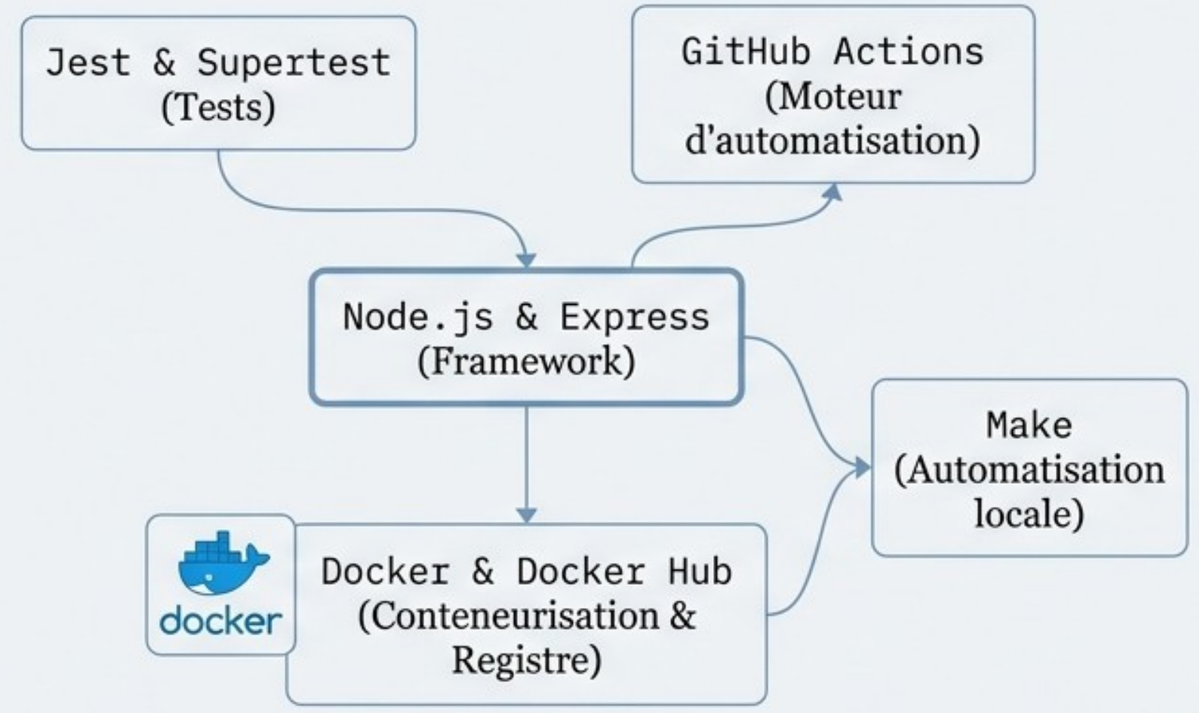
Auteurs : Anas Haddou & Amine Mohammed TAyek
Encadré par : Pr. Mohtaram
Filière : Ingénierie Informatique (Année 2025-2026)

NotebookLM

Objectifs du Projet

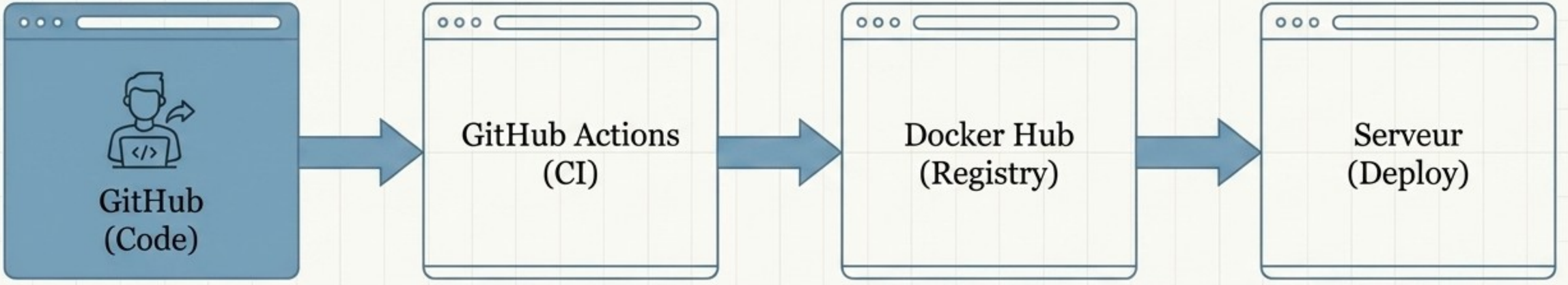
- ✅ Développement : Application web testée automatisée
- ✅ Conteneurisation : Stratégie Docker optimisée et portable
- ✅ Automatisation : Pipeline CI/CD sans intervention humaine

La Stack Technologique



NotebookLM

L'Architecture du Système



Action : Push

Action : Déclenchement des tests et build

Action : Stockage de l'image

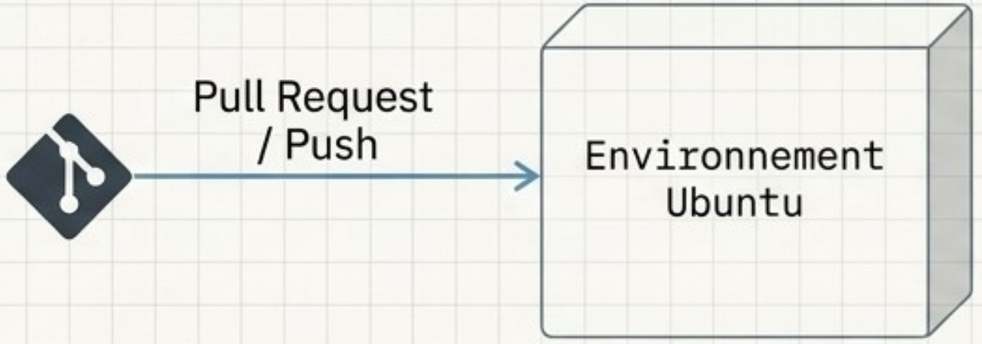
Action : Récupération et déploiement continu

Glossaire contextualisé : Pipeline - Suite d'étapes automatisées pour livrer un logiciel.

NotebookLM

Phase 1 : Intégration Continue (CI)

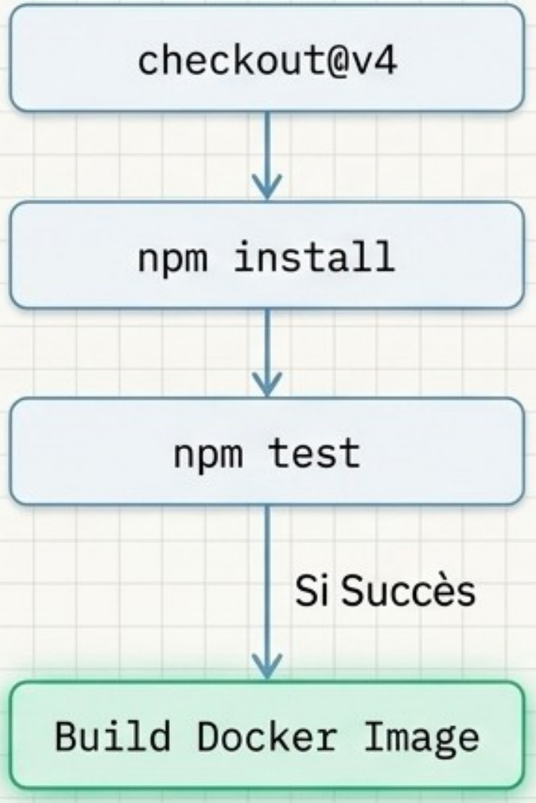
Le Déclencheur



Couverture de test à 100% sur les endpoints critiques

Glossaire : CI (Continuous Integration) - Pratique de fusionner les modifications de code fréquemment dans un dépôt central.

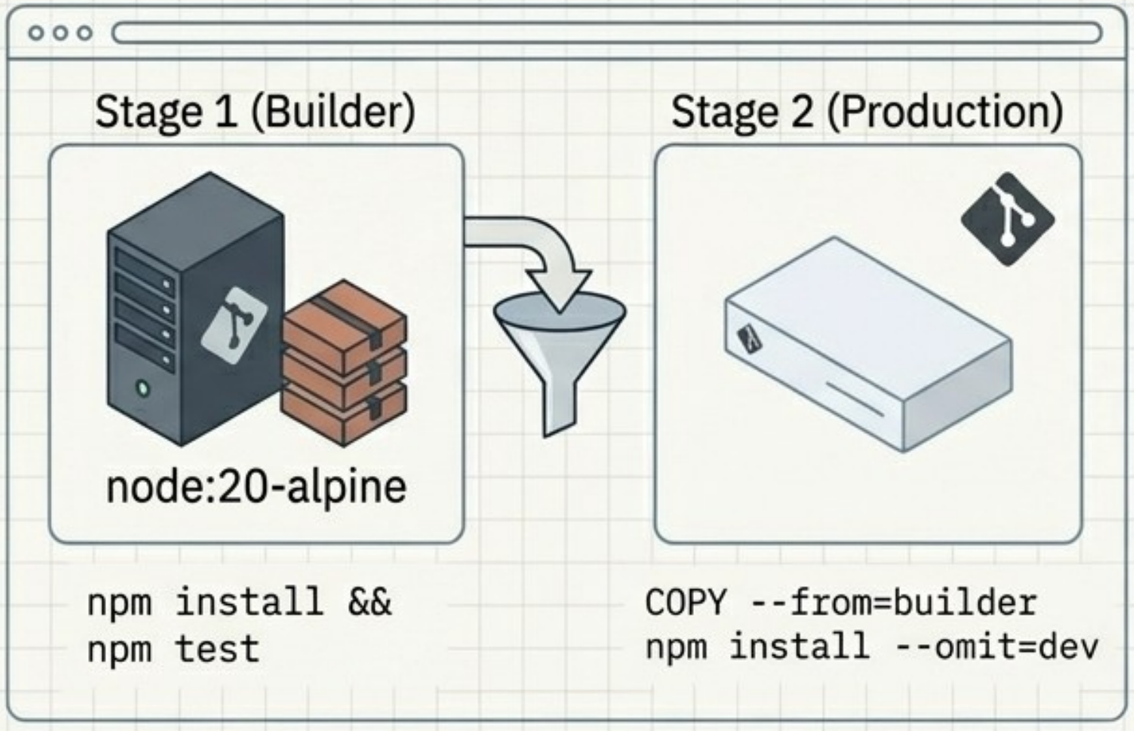
Logique du Workflow (`main.yml`)



NotebookLM

Phase 2 : Stratégie de Conteneurisation Optimisée

L'Architecture Multi-stage



Approche Classique	Approche Optimisée (Multi-stage)
Image lourde, surface d'attaque large.	Image minimale, empreinte de sécurité réduite.
Taille : ~900 Mo.	Taille : ~100 Mo.

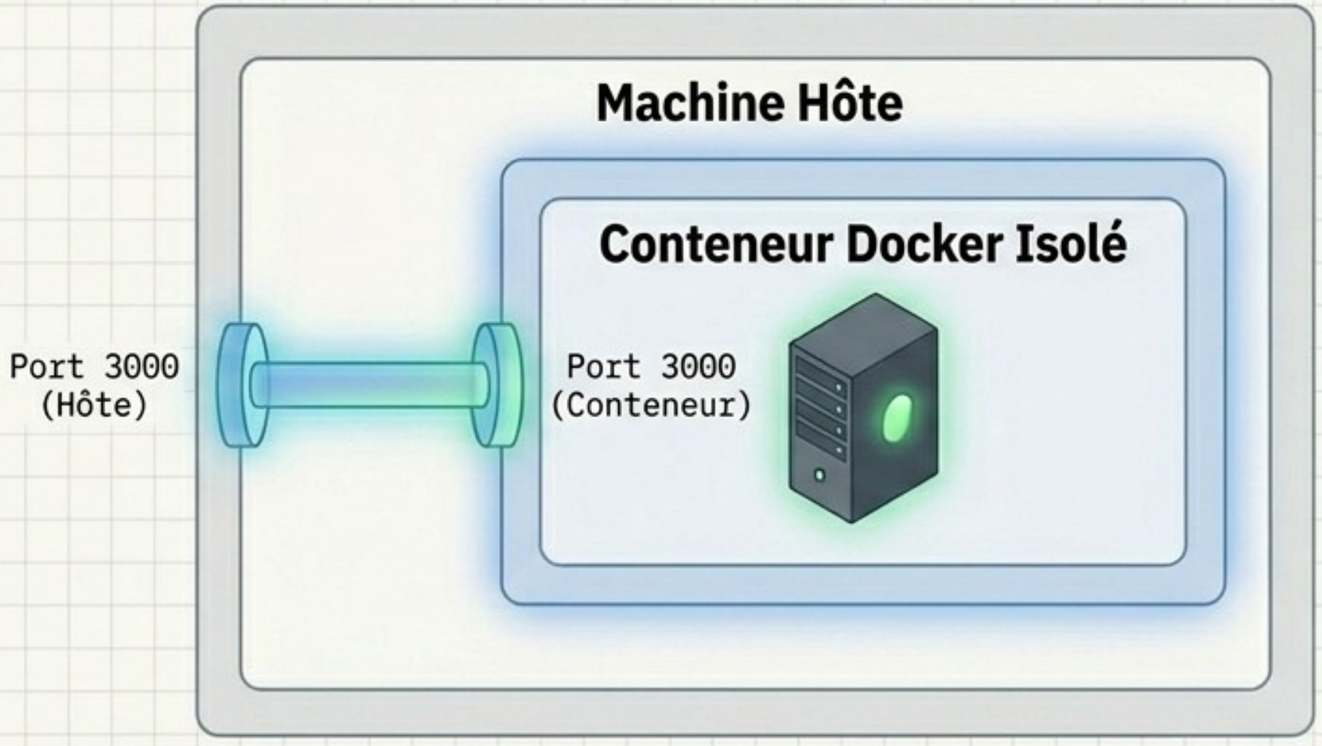
Réduction spectaculaire de 88%

Glossaire : Conteneur - Unité standard de logiciel qui empaquette le code et ses dépendances.

NotebookLM

Phase 3 : Déploiement Continu Local

L'Environnement Docker Compose



Isolation : Service indépendant et encapsulé.

Fiabilité : Politique de redémarrage (`restart: always`).

Accessibilité : Disponible via `http://localhost:3000` avec réponse JSON formatée.

Glossaire : CD (Continuous Deployment) - Automatisation du déploiement de chaque modification validée en production.

NotebookLM

Normes de Sécurité et Architecture



Gestion des Secrets

Utilisation des GitHub Secrets pour chiffrer les identifiants Docker Hub (empêchant les fuites dans le code source).



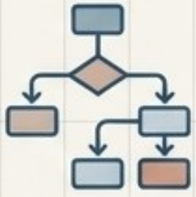
Minimisation de la Surface d'Attaque

Utilisation exclusive d'images Alpine minimalistes.



Validation Stricte

Déploiement rigoureusement conditionné par la réussite préalable de 100% des tests unitaires.



Séparation des Responsabilités

Refactoring de l'architecture (/routes, /controllers, /services) pour une maintenabilité accrue.

NotebookLM

Observabilité : Dashboard DevOps

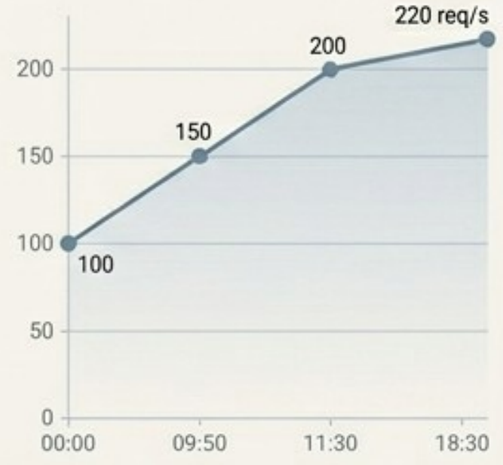
Statut Global (GET /api/health)

UP

Response Time: 0.10s

Métriques (GET /api/metrics)

Requêtes 220 req/s



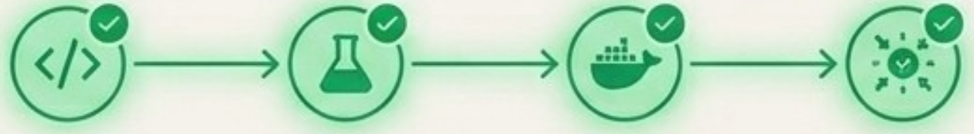
Uptime du Système 99.99% Uptime



Métadonnées de Build (GET /api/builld-info)

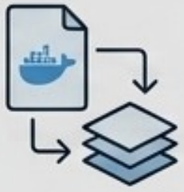
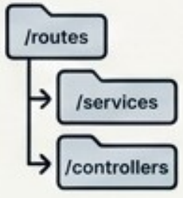
Hash Git: 7f8e4a5b
Version: 1.2.3-stable
Date du dernier déploiement: 2024-05-22 14:35 UTC

Statut du Pipeline CI/CD



NotebookLM

Rétrospective : Défis et Solutions

Défi (Problème)	Solution Technique	Impact Obtenu
 Fuite potentielle d'identifiants `Docker Hub` dans le code.	 Implémentation des `GitHub Actions Secrets`.	 Déploiement sécurisé sans credentials en clair.
 Images Docker excessivement lourdes (**900 Mo**).	 Fichier `.dockerignore` et **Builds Multi-étapes**.	 Réduction de l'empreinte à **100 Mo** et temps de build accéléré.
 Architecture monolithique empêchant l'évolutivité.	 Restructuration complète (dossiers `/routes`, `/services`, etc.).	 Code lisible, maintenable et hautement testable.

NotebookLM

Résultats et Synthèse

KPI highlight dashboard

100%

Couverture de tests unitaires sur les `endpoints` critiques.

< 2 Min

Vitesse d'exécution du `pipeline` de bout en bout.

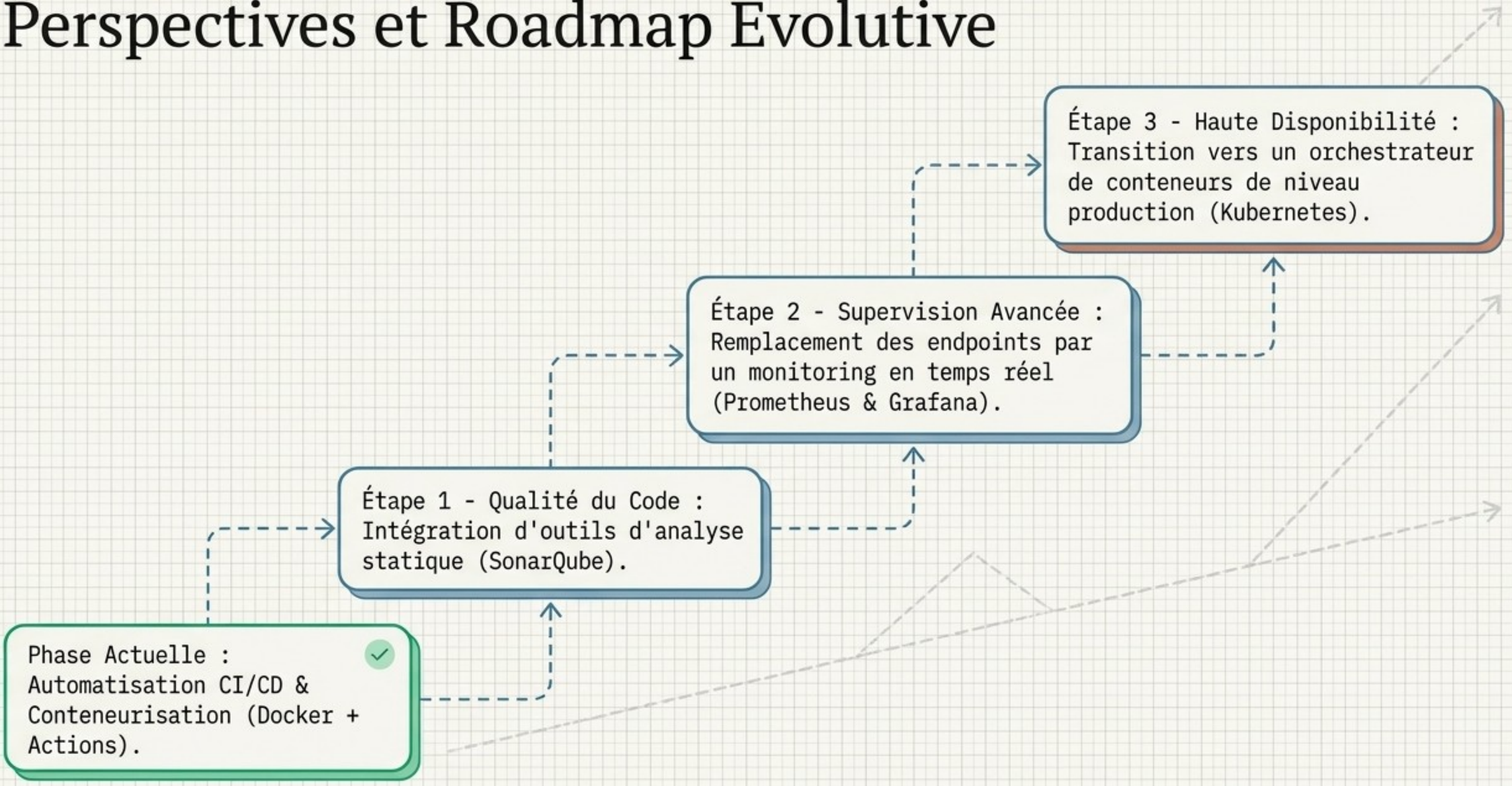
88%

Réduction de la taille de l'image (de `900 Mo` à `100 Mo` via `Alpine`).

L'intégration conjointe de `Docker` et `GitHub Actions` élimine les erreurs humaines et transforme une application locale en une solution Dev0ps de bout en bout, fiable et automatisée.

NotebookLM

Perspectives et Roadmap Évolutive



NotebookLM